
Editor de mallas 3D
3D Mesh editor



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Sebastián Sánchez Hombría

Director

Rubén Rafael Rubio Cuéllar

Grado en Desarrollo de Videojuegos

Facultad de Informática

Universidad Complutense de Madrid

Editor de mallas 3D 3D Mesh editor

Trabajo de Fin de Grado en Desarrollo de Videojuegos

Autor

Sebastián Sánchez Hombría

Director

Rubén Rafael Rubio Cuéllar

Convocatoria: *Septiembre 2026*

Grado en Desarrollo de Videojuegos

Facultad de Informática

Universidad Complutense de Madrid

DIA de MES de AÑO

Dedicatoria

A rellenar rápido

Agradecimientos

A Guillermo, por el tiempo empleado en hacer estas plantillas. A Adrián, Enrique y Nacho, por sus comentarios para mejorar lo que hicimos. Y a Narciso, a quien no le ha hecho falta el Anillo Único para coordinarnos a todos.

Resumen

Editor de mallas 3D

Editor de mallas 3D es un proyecto cuyo objetivo principal es ayudar en el proceso de edición de una malla tridimensional, con particular énfasis en la simplificación de la tarea de aportar volumen con facilidad, y construir y destruir topología básica, sin necesidad de tener conocimiento en el manejo de herramientas de modelado 3D tradicionales más avanzadas, como Blender o Maya.

Palabras clave

Editor, malla, 3D, modelado

Abstract

3D Mesh editor

An abstract in English, half a page long, including the title in English. Below, a list with no more than 10 keywords.

Keywords

10 keywords max., separated by commas.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	3
1.4. Estructura de la memoria	3
2. Estado de la cuestión	5
2.1. Herramientas de edición de mallas existentes	5
2.1.1. Historia	5
2.1.2. Blender	6
2.1.3. Autodesk Maya	6
2.1.4. Motivación de desarrollo frente a otras opciones	6
2.2. Tecnologías utilizadas	6
2.2.1. OpenGL	6
2.2.2. glad	7
2.2.3. glm	8
2.2.4. Qt	8
2.2.5. Mallas	8
2.2.6. Almacenamiento de mallas	9
2.3. Conceptos y técnicas de gráficos 3D	9
2.3.1. Pipeline gráfico	9
2.3.2. Pipeline de coordenadas	10
2.3.3. Buffers de geometría en OpenGL	11
2.3.4. Selección de elementos en aplicaciones 3D	12
2.3.5. Raycasting	12

3. Descripción del trabajo	15
3.1. Estructura del programa	15
3.1.1. Interfaz de usuario	15
3.1.2. Lógica del programa	16
3.2. Funcionalidades	17
3.2.1. Importación y exportación de modelos	17
3.2.2. Visualización de la escena	17
3.2.3. Selección de elementos	19
3.2.4. Edición de la malla	19
3.2.5. Atajos de teclado	22
3.3. Detalles de implementación	22
3.3.1. Sistema de renderizado	22
3.3.2. Representación de la malla	26
3.3.3. Selección	30
3.3.4. Cámara	33
3.3.5. Operaciones de manipulación	35
3.3.6. Ratón y teclado	38
4. Conclusiones y trabajo futuro	41
4.1. Conclusiones	41
4.2. Trabajo futuro	42
4.2.1. Rendimiento	42
4.2.2. Funcionalidad	42
4.2.3. Robustez	43
4.2.4. Extensiones	43
Introduction	45
Conclusions and Future Work	47
Bibliografía	49

Índice de figuras

2.1. Pipeline de transformación de coordenadas.	11
3.1. Estructura del programa.	17
3.2. Interfaz de usuario de la aplicación.	18
3.3. Controles de la cámara.	19
3.4. Comparación entre proyección perspectiva/ortogonal, y renderizado sólido/wireframe.	20
3.5. Selección de vértices, segmentos y polígonos	20
3.6. Tipos de transformaciones	21
3.7. Extrusión y eliminacion de caras seleccionadas.	21
3.8. Tres vértices de renderizado se manipulan con uno lógico.	29
3.9. Triangulación de polígono en abanico.	33

Índice de tablas

3.1. Atajos de teclado del editor.	22
--	----

Introducción

En este capítulo se presentan el contexto y las motivaciones que han dado lugar al desarrollo de este proyecto. Asimismo, se describen los objetivos planteados y el enfoque seguido para su compleción.

El proyecto se plantea tanto como el desarrollo de una herramienta de edición como un estudio de las operaciones que pueden resultar necesarias en un editor orientado a la modificación sencilla de modelos tridimensionales. De este modo, se pretende analizar qué operaciones de transformación y edición resultan adecuadas para permitir a usuarios con experiencia principalmente en herramientas bidimensionales realizar modificaciones sobre modelos 3D sin necesidad de disponer de conocimientos avanzados sobre software de modelado.

1.1. Motivación

La generación y transformación de contenido bidimensional en modelos tridimensionales constituye un área ampliamente estudiada dentro de los gráficos de computador. Uno de los principales objetivos de este tipo de técnicas es facilitar la creación de mallas 3D a partir de contenido bidimensional, permitiendo que usuarios familiarizados con herramientas y procesos de trabajo 2D puedan obtener representaciones tridimensionales sin necesidad de adquirir todos los conocimientos asociados al modelado y desarrollo de contenido 3D.

En la actualidad existen diferentes herramientas capaces de generar modelos tridimensionales a partir de imágenes bidimensionales. Sin embargo, el resultado obtenido mediante estos procesos automáticos no siempre coincide con las expectativas del usuario, y en estas situaciones puede resultar necesario realizar modificaciones sobre el modelo generado. No obstante, se carece de herramientas que permitan a estos mismos usuarios realizar transformaciones sencillas sobre dichos resultados cuando estos no son exactamente los esperados.

La edición de modelos tridimensionales suele requerir el uso de aplicaciones especializadas, cuyo manejo puede implicar un conocimiento previo de conceptos como la

geometría de mallas, los sistemas de coordenadas o las diferentes técnicas de transformación y modelado. Esta complejidad puede suponer una barrera para aquellos usuarios cuyo ámbito de trabajo habitual se encuentra principalmente en el entorno bidimensional y que únicamente necesitan realizar modificaciones sencillas sobre un modelo 3D previamente generado.

A partir de esta necesidad surge de este proyecto. Se plantea el estudio de las operaciones básicas que pueden formar parte de un editor de mallas tridimensionales orientado a la realización de transformaciones sencillas. El objetivo consiste en analizar y proporcionar un conjunto de operaciones que permitan modificar fácilmente los resultados obtenidos a partir de procesos de transformación de 2D a 3D.

De esta manera, se busca reducir la complejidad asociada a la edición tridimensional y proporcionar una interfaz que permita al usuario interactuar con el modelo mediante operaciones sencillas, manteniendo al mismo tiempo la utilidad necesaria para adaptar el resultado generado.

1.2. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es estudiar y desarrollar un conjunto de operaciones de edición para un editor de mallas tridimensionales orientado a usuarios que no disponen necesariamente de conocimientos avanzados sobre modelado o desarrollo 3D.

Con este fin se establecen los siguientes objetivos:

- Estudiar las características y necesidades asociadas a la edición de modelos tridimensionales generados a partir de contenido bidimensional.
- Analizar qué operaciones de transformación y edición pueden resultar más útiles para realizar modificaciones sencillas sobre una malla tridimensional.
- Diseñar una herramienta que permita aplicar dichas operaciones de manera intuitiva.
- Implementar un sistema de visualización e interacción con modelos tridimensionales.
- Permitir la selección y manipulación de los elementos necesarios de una malla para llevar a cabo las operaciones de edición definidas.
- Integrar las operaciones estudiadas de manera que se facilite la modificación de los modelos obtenidos.

El proyecto se centra, por tanto, en el estudio y desarrollo de las operaciones necesarias para realizar modificaciones sencillas sobre modelos tridimensionales y así desarrollar una herramienta que permita intervenir sobre mallas tridimensionales generadas previamente cuando estos requieran ajustes adicionales.

1.3. Plan de trabajo

Para la consecución de los objetivos planteados, el desarrollo del proyecto se ha estructurado en varias fases de trabajo.

En primer lugar, se realizará un estudio del problema y contexto en el que se desarrolla la aplicación. Se analizarán las características de los modelos tridimensionales que deben ser tratados y estudiarán las diferentes operaciones de edición que puedan resultar de utilidad. Asimismo, se llevará a cabo una revisión de los conceptos necesarios para la representación, interacción y manipulación con mallas tridimensionales.

A continuación, se realizará un análisis de las operaciones que deberá proporcionar el editor, que permitirá determinar qué mecanismos de edición resultan adecuados para el propósito de la herramienta. Se estudiará la forma en que dichas operaciones pueden aplicarse sobre una malla y cómo deben presentarse al usuario para facilitar su utilización.

Posteriormente, se abordará el diseño de la aplicación. En esta fase se definirán la arquitectura general del sistema, la organización de los diferentes componentes y los mecanismos necesarios para el manejo de los modelos tridimensionales. También se diseñará la interacción del usuario con el editor y la forma en que se aplicarán las distintas operaciones sobre las mallas.

Finalmente se procederá a la implementación de la herramienta. El desarrollo se realizará de forma progresiva, comenzando por las funcionalidades necesarias para la carga y visualización de los modelos y continuando con la implementación de los mecanismos de interacción y de las diferentes operaciones de edición estudiadas.

1.4. Estructura de la memoria

El resto de esta memoria se organiza en los siguientes capítulos. El capítulo 2 presenta las tecnologías empleadas en el desarrollo del proyecto, así como los conceptos y técnicas de gráficos por computador en los que se apoya, y sitúa el trabajo frente a herramientas de edición de mallas 3D ya existentes. El capítulo 3 describe la arquitectura general del programa, las funcionalidades que ofrece, y los detalles de su implementación. Finalmente, el capítulo 4 valora el grado de cumplimiento de los objetivos planteados y recoge las limitaciones y posibles líneas de mejora del proyecto.

Se puede encontrar un enlace al repositorio aquí:

— INSERTAR ENLACE REPO —

Capítulo 2

Estado de la cuestión

—Introducción breve—

2.1. Herramientas de edición de mallas existentes

En esta sección se explica brevemente la historia y algunas de las alternativas de editores de mallas existentes, y el motivo del desarrollo del programa frente al uso de estas.

2.1.1. Historia

El origen de la edición interactiva de gráficos por computador se remonta a Sketchpad, desarrollado por Ivan Sutherland en 1963, un programa que permitió cambiar la comunicación con los ordenadores de instrucciones escritas, a poder interactuar mediante dibujos y manipulaciones de objetos directamente sobre una pantalla (Sutherland, 2003; Ferreira et al., 2025).

Las primeras herramientas comerciales de modelado 3D surgieron en la década de 1980. Wavefront Technologies, fundada en 1984 por Bill Kovacs, Larry Barel y Mark Sylvester desarrolló *The Advanced Visualizer*, un conjunto de programas para la creación de gráficos por computador, para el cual definió el formato OBJ (sección ??). De forma paralela, Alias Research, que pasaría posteriormente a denominarse Alias Systems Corporation antes de ser adquirida por Autodesk en 2006, desarrolló PowerAnimator, una herramienta de modelado y animación usada ampliamente en la industria del cine. En 1995, ambas compañías fueron adquiridas por Silicon Graphics y fusionadas en Alias|Wavefront, cuya tecnología combinada dio lugar en 1998 al lanzamiento de Maya (Retro Reversing, 2026).

De forma independiente, Ton Roosendaal escribió en enero de 1994 el código fuente de lo que llegaría a ser Blender, inicialmente como herramienta interna del estudio de animación NeoGeo. En octubre de 2002, tras el cierre de la compañía que lo comercializaba (Not a Number), se publicó como software libre mediante una

campana de recaudación que permitió comprar los derechos sobre su código fuente, y procedería a convertirse en una de las herramientas de edición de mallas más reconocidas a día de hoy (Blender Foundation, 2025).

2.1.2. Blender

Blender es un conjunto de herramientas de creación 3D de código abierto que cubre modelado, esculpido, animación, simulación, renderizado y composición dentro de una única aplicación (Blender Foundation, 2026a). Su modo de edición de mallas (*Edit Mode*) trabaja con los mismos tres tipos de elementos seleccionables que este proyecto (vértices, aristas y caras), y ofrece un control explícito sobre el comportamiento de operaciones como la eliminación de elementos, permitiendo al usuario decidir cuánta geometría conectada desea conservar (sección 3.3.5.2).

2.1.3. Autodesk Maya

Maya es una herramienta de modelado y animación 3D de uso extendido en la industria del cine y los videojuegos, heredera directa de la tecnología de Alias|Wavefront. A diferencia de Blender, su modelo de licencia es comercial, y su conjunto de herramientas de edición de mallas está pensado para integrarse dentro de *pipelines* de producción más amplios (Autodesk Inc, 2026).

2.1.4. Motivación de desarrollo frente a otras opciones

Frente a Blender y Maya, herramientas de propósito general con décadas de desarrollo detrás, el editor desarrollado en este trabajo se plantea como una herramienta de propósito acotado, centrada específicamente en las operaciones fundamentales de edición de mallas poligonales: selección, transformación, eliminación y extrusión. Esto permite dedicar el trabajo a comprender e implementar en profundidad los fundamentos de estas operaciones.

2.2. Tecnologías utilizadas

A continuación se describen brevemente las distintas tecnologías utilizadas en el transcurso del proyecto y el motivo de su uso.

2.2.1. OpenGL

Para desarrollar una aplicación gráfica, el proyecto requiere una API gráfica que permita renderizar modelos tridimensionales, gestionar *shaders* y mantener la posibilidad de ejecutar la aplicación en distintos sistemas operativos.

OpenGL (*Open Graphics Library*) es una especificación abierta y multiplataforma para el desarrollo de aplicaciones que produzcan gráficos 2D y 3D. Fue desarrollada originalmente por Silicon Graphics Inc. y publicada por primera vez en 1992. Desde 2006, su desarrollo y mantenimiento corren a cargo de Khronos Group, un consorcio industrial fundado en el año 2000, que ha continuado publicando nuevas versiones de la especificación hasta la actualidad (Khronos Group, 2026a, 2022).

Para el proyecto se consideran varias alternativas comunes para aplicaciones gráficas: Direct3D, Vulkan y OpenGL. Direct3D es una API gráfica propietaria desarrollada por Microsoft, disponible únicamente en Windows. Vulkan, gestionada también por Khronos Group, es una API de bajo nivel que ofrece un control explícito sobre los recursos de la GPU (gestión de memoria, sincronización de comandos, etc.), a cambio de una mayor complejidad en el código. OpenGL, en cambio, ofrece un nivel de abstracción más alto, delegando en el controlador gráfico buena parte de la gestión de recursos, y cuenta con una documentación extensa y numerosas bibliotecas de utilidad desarrolladas por la comunidad.

La primera consideración para escoger el candidato adecuado fueron las capacidades multiplataforma de la API. Debido a que Direct3D únicamente ofrece soporte para Windows, queda descartada como opción. Entre OpenGL y Vulkan, se opta por OpenGL: su curva de aprendizaje más reducida, su documentación extensa y el acceso a numerosas bibliotecas de utilidad la convierten en la opción más apropiada para el proyecto. A esto se suma la familiaridad previa con OpenGL, adquirida durante el grado, lo que facilita su uso durante el desarrollo del proyecto.

2.2.2. glad

En la mayoría de plataformas, el sistema operativo solo expone de forma directa (estática) las funciones correspondientes a OpenGL 1.1. El acceso a las funciones introducidas en versiones posteriores, así como a las extensiones específicas del controlador gráfico, requiere obtener el puntero a cada función en tiempo de ejecución, una vez creado el contexto de OpenGL, mediante mecanismos propios de cada plataforma (por ejemplo, `wglGetProcAddress` en Windows o `glXGetProcAddress` en Linux) (Khronos Group, 2026b). Realizar este proceso manualmente para cada función resulta tedioso y propenso a errores, además de obligar a escribir código distinto para cada sistema operativo, lo cual compromete la portabilidad buscada en el proyecto.

Para evitar este trabajo repetitivo, se usa glad, una biblioteca que genera automáticamente, a partir de las especificaciones oficiales de Khronos Group, el código necesario para cargar en tiempo de ejecución los punteros a las funciones compatibles con la versión de OpenGL utilizada, abstrayendo además las diferencias entre plataformas.

2.2.3. glm

Un requisito de un editor de mallas es poseer la capacidad de realizar operaciones matriciales, alternar entre los distintos sistemas de coordenadas empleados en la representación tridimensional (modelo, vista y proyección), y transformar coordenadas de pantalla a coordenadas del mundo (por ejemplo, para implementar la selección de elementos mediante *ray casting*), entre otras. Con este fin, glm es una biblioteca que se adapta a las necesidades de este trabajo, ofreciendo las operaciones matemáticas de vectores y matrices en las que se apoyan este tipo de funcionalidades en aplicaciones gráficas (G-Truc, 2026).

2.2.4. Qt

Qt es un framework multiplataforma para el desarrollo de interfaces gráficas, originado en 1991 de la mano de Haavard Nord y Eirik Chambe-Eng, cuya primera versión pública se publicó en 1995 bajo la empresa Trolltech. Desde entonces, su desarrollo ha continuado bajo distintas compañías, siendo en la actualidad mantenido por The Qt Company junto con la comunidad de código abierto del Qt Project. Qt posee capacidad de creación de ventanas y elementos de interfaz de usuario (Qt Group, 2026a,b).

El proyecto requiere una ventana de renderizado, además de gestión del contexto de OpenGL y del input del usuario. Para la creación del contexto de OpenGL y de la ventana se consideró GLFW, una biblioteca multiplataforma que ofrece una API sencilla centrada exclusivamente en estos fines, y que constituye una alternativa habitual a frameworks más completos como Qt cuando no se necesita un sistema de interfaz de usuario elaborado. Qt, por su parte, además de estas capacidades básicas, incorpora un extenso conjunto de widgets y herramientas para la construcción de interfaces gráficas complejas (menús, paneles, diálogos, etc.).

Dado que el proyecto, más allá de la ventana de renderizado, requiere una interfaz de usuario con distintos paneles y controles para la edición de mallas, las herramientas de creación de elementos de interfaz que ofrece Qt fueron el motivo de su elección frente a GLFW.

2.2.5. Mallas

Una malla poligonal es la estructura de datos habitual para representar la superficie de un objeto tridimensional. Está formada por un conjunto de vértices, cada uno de los cuales almacena, como mínimo, una posición en el espacio, y opcionalmente otros atributos como su normal o sus coordenadas de textura. Los vértices se conectan mediante aristas, y los conjuntos de aristas que forman un contorno cerrado delimitan una cara, la unidad básica que compone la superficie del objeto.

Aunque una cara puede estar formada, en principio, por cualquier número de vértices, en gráficos por computador en tiempo real las mallas se representan casi siempre mediante caras triangulares: al ser un polígono siempre plano y convexo, un

triángulo no presenta ambigüedades a la hora de ser rasterizado, lo que simplifica su procesamiento por parte de la GPU. Aquellas mallas que originalmente contienen caras con más de tres vértices (por ejemplo, cuadriláteros) se descomponen en triángulos mediante un proceso de triangulación antes de ser renderizadas.

En cuanto a su representación en memoria, una malla se almacena habitualmente como una lista de vértices junto con una lista de caras, donde cada cara no repite la información de sus vértices, sino que hace referencia a ellos mediante sus índices dentro de la lista de vértices, dando lugar a lo que se conoce como malla indexada (sección 2.2.6).

2.2.6. Almacenamiento de mallas

Existen numerosos formatos de archivo para el almacenamiento de mallas tridimensionales, cada uno orientado a distintos casos de uso: formatos como FBX o glTF permiten representar escenas completas, con jerarquías de objetos, animaciones y materiales complejos, mientras que otros, como OBJ (*Wavefront Object*), se centran en representar de forma sencilla la geometría de una malla individual (vértices, normales, coordenadas de textura y caras), junto con sus materiales básicos en un archivo asociado (MTL). Por su simplicidad y su amplia adopción como formato de intercambio, OBJ es habitual como formato de entrada en herramientas de edición de mallas centradas en la geometría, sin necesidad de gestionar información adicional de escena o animación.

Para importar estos archivos a la herramienta de edición se necesita una biblioteca de carga. Una opción popular es Assimp, que permite importar archivos en numerosos formatos, frente al formato único que soporta tinyobjloader (OBJ).

Tinyobjloader ofrece tanto una integración más sencilla como una menor complejidad de uso, por lo que resulta la mejor opción para el proyecto, junto con el formato OBJ.

2.3. Conceptos y técnicas de gráficos 3D

2.3.1. Pipeline gráfico

El *pipeline* gráfico (*graphics pipeline*) es la secuencia de etapas mediante la cual una GPU transforma los datos de una escena tridimensional en una imagen bidimensional. Aunque su implementación concreta varía entre APIs, sus etapas principales son las siguientes.

En primer lugar, el *vertex shader* procesa cada vértice de la malla de forma individual, aplicando sobre él las transformaciones necesarias para llevarlo desde su sistema de coordenadas local hasta el espacio de recorte (sección 2.3.2). A continuación, la etapa de ensamblado de primitivas (*primitive assembly*) agrupa los vértices procesados en primitivas geométricas (triángulos), y realiza el recorte (*clipping*) de

aquellas que quedan parcial o totalmente fuera del volumen visible, así como, opcionalmente, el descarte de caras no orientadas hacia la cámara (*culling*).

Estas primitivas pasan después por la etapa de rasterización, que las convierte en un conjunto de fragmentos: unidades discretas asociadas a las posiciones de píxel que la primitiva cubre en pantalla, e interpolando entre los vértices los datos asociados a cada una (color, normales, coordenadas de textura, etc.). Cada fragmento es procesado individualmente por el *fragment shader*, encargado de calcular su color final, típicamente a partir de cálculos de iluminación y texturizado. Por último, los fragmentos resultantes son sometidos a una serie de operaciones por fragmento (test de profundidad, de *stencil*, mezcla de color, entre otras) que determinan si son escritos en el *framebuffer* y cómo se combinan con el contenido ya existente en él, dando lugar a la imagen final.

Tanto el *vertex shader* como el *fragment shader* están escritas en un lenguaje de *shading*, como GLSL en el caso de OpenGL, lo que permite personalizar el comportamiento del *pipeline* para cada aplicación.

2.3.2. Pipeline de coordenadas

Para representar en una pantalla bidimensional un elemento que existe en un espacio tridimensional, es necesario aplicar sobre él una secuencia de transformaciones, cada una de las cuales lo lleva de un sistema de coordenadas al siguiente. Estas transformaciones son las que se aplican durante la etapa de *vertex shader* del *pipeline* gráfico (sección 2.3.1).

Un objeto se describe en unas coordenadas locales, definidas respecto al origen que tenía en el momento de su creación. Multiplicando la posición de cada vértice por una matriz de modelado, se obtienen sus coordenadas globales o de mundo: el espacio compartido en el que coexisten una o más mallas, cada una posicionada respecto a un mismo origen común.

A continuación, se aplica una matriz de vista, que transforma dichas coordenadas de mundo a coordenadas relativas a una cámara: una abstracción definida por una posición y una orientación, que determina desde dónde y hacia dónde se observa la escena. Es en este paso donde se obtiene una representación de cómo se observarían los objetos del mundo desde la perspectiva de dicha cámara. La cámara, como cualquier otro elemento de la escena, puede trasladarse y rotar libremente dentro del mundo; sin embargo, en su propio sistema de referencia permanece siempre situada en el origen y orientada en una dirección fija, siendo el resto del mundo el que se transforma en relación a ella al aplicar la matriz de vista.

A continuación, se aplica una matriz de proyección, que transforma dichas coordenadas de cámara al espacio de recorte (*clip space*). Esta matriz determina el volumen de la escena que resulta visible, descartando aquello que quede fuera de él y difiere según el tipo de proyección empleada, en perspectiva u ortogonal, y lo proyecta sobre un plano.

Las coordenadas de recorte no son directamente utilizables: es necesario dividir

sus tres primeras componentes entre su cuarta componente, un paso conocido como división de perspectiva, para obtener las coordenadas normalizadas de dispositivo (*normalized device coordinates*, NDC), en las que la escena visible queda representada dentro de un cubo de lado 2 centrado en el origen (las coordenadas van desde -1 a 1). Finalmente, mediante la transformación de *viewport*, estas coordenadas normalizadas se convierten en coordenadas de pantalla (*screen space*), expresadas en píxeles y referidas a las dimensiones reales de la ventana de renderizado (figura 2.1).

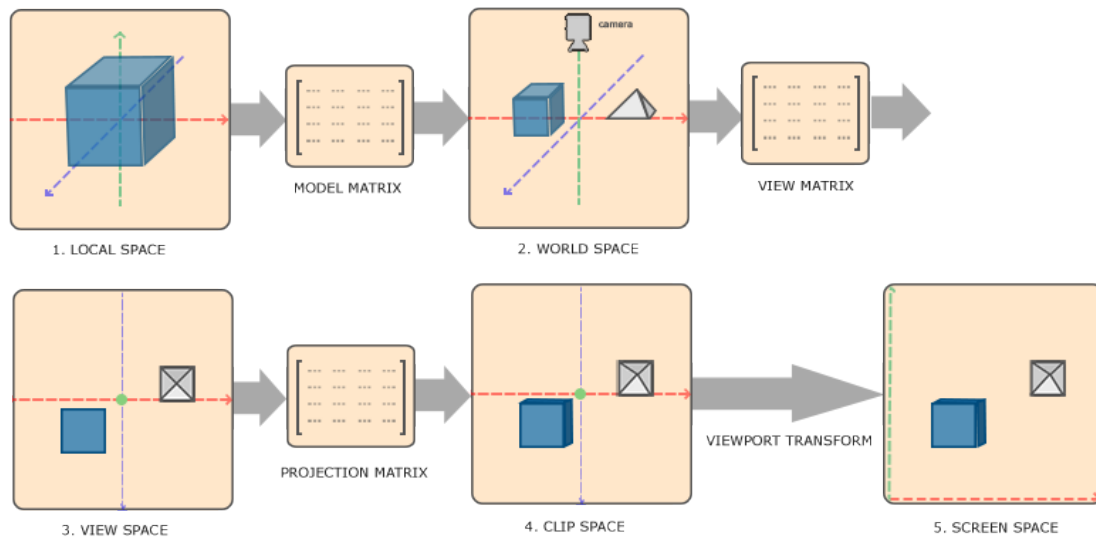


Figura 2.1: Pipeline de transformación de coordenadas.
(de Vries, s.f.a)

2.3.3. Buffers de geometría en OpenGL

En el *core profile* de OpenGL utilizado en aplicaciones modernas no es posible renderizar geometría directamente desde estructuras de datos en memoria de la CPU: es necesario almacenar previamente dicha información en memoria de la GPU, mediante objetos específicos para este fin. Esto supone un cambio respecto al llamado modo inmediato (*immediate mode*), disponible en versiones antiguas de OpenGL y en el *compatibility profile*, en el que los vértices podían especificarse y transmitirse a la GPU en cada frame. Dicho enfoque quedó obsoleto por su impacto negativo en el rendimiento, especialmente en el caso de geometría estática.

Para transmitir y organizar la geometría de una malla se emplean habitualmente tres tipos de objetos:

- **Vertex Buffer Object (VBO):** almacena los atributos de los vértices (posición, normal, coordenadas de textura, entre otros) que serán utilizados durante el renderizado. Al cargar esta información una única vez en memoria de GPU, se evitan transferencias repetidas entre CPU y GPU en cada frame.
- **Element Buffer Object (EBO),** también conocido como *index buffer*: almacena una lista de índices que hacen referencia a los vértices del VBO, en lugar

de repetir sus atributos por cada triángulo que los utiliza. Esta técnica de renderizado, en la que la geometría se referencia mediante índices en vez de duplicar vértices, se conoce como **mallá indexada**.

- *Vertex Array Object* (VAO): almacena la configuración necesaria para que la GPU sea capaz de interpretar los atributos contenidos en el VBO: cuántos atributos hay, de qué tipo son, y en qué posición de memoria se encuentra cada uno dentro de la estructura de cada vértice.

El conjunto formado por un VAO junto con el VBO (y, en su caso, el EBO) que referencia constituye la representación en GPU de una mallá, a la que se hace referencia mediante el identificador de dicho VAO en las llamadas de renderizado.

2.3.4. Selección de elementos en aplicaciones 3D

En aplicaciones interactivas de edición 3D es habitual que el usuario deba señalar con el ratón un elemento concreto de una escena tridimensional, un proceso conocido como *picking*. Existen distintas técnicas para resolver este problema, que difieren en el espacio en que se realiza la comparación y en su coste computacional.

El *raycasting* consiste en proyectar un rayo desde la posición del ratón hacia la escena, y determinar con qué elementos de la geometría dicho rayo intersecta. Una alternativa consiste en proyectar la geometría tridimensional al espacio de pantalla y comparar directamente en dos dimensiones frente a la posición del ratón, evitando el cálculo de intersecciones del rayo y la geometría en tres dimensiones.

Una tercera técnica, conocida como *back-buffer selection* o selección mediante búfer auxiliar, evita por completo el cálculo geométrico de proximidad: la escena se renderiza en un búfer fuera de pantalla, asignando a cada elemento seleccionable un color plano único que codifica su identificador, y la selección se resuelve leyendo el color del píxel bajo el cursor y decodificándolo de vuelta a dicho identificador (Kreuzer, 2026). Esta técnica ofrece una selección exacta a nivel de píxel sin necesidad de ningún cálculo de distancia o intersección, a cambio de requerir una pasada de renderizado adicional y una lectura síncrona del contenido de la GPU hacia la CPU (`glReadPixels`), una operación que puede introducir una espera hasta que la GPU termine de procesar dicha pasada.

2.3.5. Raycasting

El *raycasting* es una técnica que consiste en proyectar un rayo y determinar con qué elementos de la geometría de una escena intersecta.

En el caso de mallás poligonales, dado que estas se componen de caras triangulares, la intersección del rayo con la mallá se reduce a comprobar su intersección individual con cada uno de sus triángulos, un problema conocido como intersección rayo-triángulo. Existen distintos algoritmos para resolverlo. Uno de los más empleados es el algoritmo de Möller–Trumbore, que calcula el punto de intersección,

si existe, en términos de sus coordenadas baricéntricas respecto al triángulo, sin necesidad de precalcular la ecuación del plano que lo contiene.

El raycasting tiene aplicación en múltiples ámbitos de los gráficos por computador, como la detección de colisiones o la selección de elementos de una escena mediante el ratón, proyectando un rayo desde la posición del cursor hacia el interior de la escena y determinando el primer elemento con el que intersecta.

Capítulo 3

Descripción del trabajo

En este capítulo se describe el editor de mallas desarrollado para este trabajo. En primer lugar, se presenta a grandes rasgos la arquitectura general del programa. A continuación, se enumeran las distintas funcionalidades que ofrece, y por último se detalla cómo se ha implementado cada una de ellas.

3.1. Estructura del programa

El flujo de la aplicación para la edición de una malla es el siguiente: el modelo se carga desde un archivo en disco, y se transforma en una representación interna para su edición. Sobre esta representación, el usuario aplica una serie de operaciones de selección y transformación, que quedan representados en la visualización del modelo. Finalmente, la malla se puede exportar a un nuevo archivo en disco. La figura 3.1 ilustra este flujo, así como las dos partes principales en las que se organiza la aplicación para llevarlo a cabo.

3.1.1. Interfaz de usuario

La ventana principal de la aplicación se organiza en torno a tres elementos: un lienzo central, donde se renderiza y edita la escena tridimensional, una barra de herramientas, desde la que el usuario controla el modo de edición activo en cada momento, y desde la cual puede cargar y guardar la malla trabajada, y desplegar un panel con información adicional sobre el control de la cámara, y un panel de transformación, que muestra y permite modificar la posición en la escena del elemento actualmente seleccionado.

El lienzo ocupa la mayor parte del área central de la ventana, y constituye la superficie sobre la que se dibuja la malla y se capturan las interacciones del usuario. La barra de herramientas, en la zona superior de la ventana, agrupa las distintas acciones disponibles en botones organizados por categorías, cada una representada mediante un conjunto de botones mutuamente excluyentes: el modo de selección

(vértices, aristas o polígonos), el modo de transformación (traslación, rotación o escalado), el eje sobre el que se restringe dicha transformación, el modo de renderizado (sólido o *wireframe*) y el tipo de proyección de la cámara (perspectiva u ortogonal). El panel lateral, en el lateral derecho de la ventana, permite además ajustar numéricamente la posición de la selección actual mediante tres campos correspondientes a sus coordenadas X, Y y Z.

La interfaz de usuario se implementa mediante Qt, una biblioteca externa diseñada para el desarrollo de interfaces gráficas, encargada de construir la ventana principal y todos sus elementos (lienzo, barra de herramientas y panel lateral), así como de gestionar el menú de importación y exportación de modelos (sección 3.1.1).

Cada acción de la barra de herramientas se conecta directamente, mediante el sistema de señales de Qt, a un método de la lógica de edición que modifica su estado. Esta vía de comunicación es independiente del resto del input de la aplicación: mientras que el movimiento del ratón sobre el lienzo o la mayoría de las teclas del teclado se centralizan y consultan explícitamente en cada iteración del bucle principal (sección 3.3.6), las acciones de la barra de herramientas tienen efecto de forma inmediata en el momento en que se pulsan, ya sea con el ratón o mediante su atajo de teclado asociado.

3.1.2. Lógica del programa

La lógica del programa es responsable de mantener y modificar el estado de la malla que se está editando, y queda coordinada por la clase **Editor**, que integra las siguientes responsabilidades:

- La malla en sí, representada por la clase **Mesh**, que almacena su geometría y ofrece además las operaciones de carga y guardado del modelo desde y hacia archivo (sección 3.3.2).
- La selección de elementos de la malla, implementada por la clase **Selector** (sección 3.3.3).
- Las operaciones de edición aplicadas sobre los elementos seleccionados, definidas en la clase **MeshManipulator** (sección 3.2.4).
- La cámara, gestionada por la clase **Camera**, que determina desde qué posición y con qué proyección se observa la escena (sección 3.3.4).
- El renderizado del modelo y de los elementos seleccionados, cuya responsabilidad no recae en una única clase, sino que se reparte entre **Mesh** (que dibuja su propia geometría), los programas de shaders (sección 3.3.1.2) y **Camera** (sección 3.3.4), que aporta las matrices de vista y proyección necesarias. Es el propio **Editor** quien coordina este proceso en cada frame, combinando dichas matrices con la matriz de modelo y emitiendo las llamadas de dibujado correspondientes.

Cabe destacar que la separación entre la interfaz y la lógica del programa no es absoluta: por conveniencia, el propio **Editor** hace uso del sistema de señales de Qt para comunicar a la interfaz cambios en la posición del elemento actualmente seleccionado, y además comunica a la lógica cambios en los valores de la interfaz, de manera que la modificación de alguno de los dos afecta al otro.

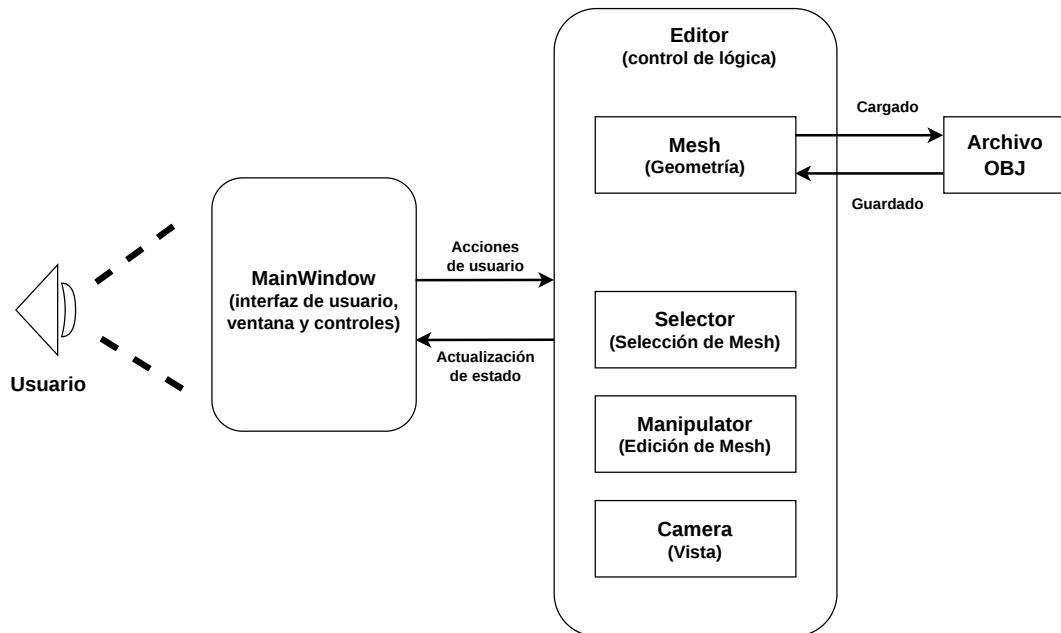


Figura 3.1: Estructura del programa.

3.2. Funcionalidades

En esta sección se describen las funcionalidades que ofrece el editor desde la perspectiva del usuario, sin entrar en detalles de su implementación, los cuales se tratan en la sección 3.3.

3.2.1. Importación y exportación de modelos

El editor permite cargar modelos tridimensionales en formato OBJ desde el menú *Archivo > Abrir*, así como guardar el estado actual de la malla editada en el mismo formato mediante *Archivo > Guardar* (figura 3.2).

3.2.2. Visualización de la escena

El usuario puede observar el modelo desde cualquier ángulo mediante el control de la cámara: manteniendo pulsado el botón central del ratón se orbita alrededor del modelo, combinándolo con la tecla **Shift** se desplaza la vista lateralmente (paneo),

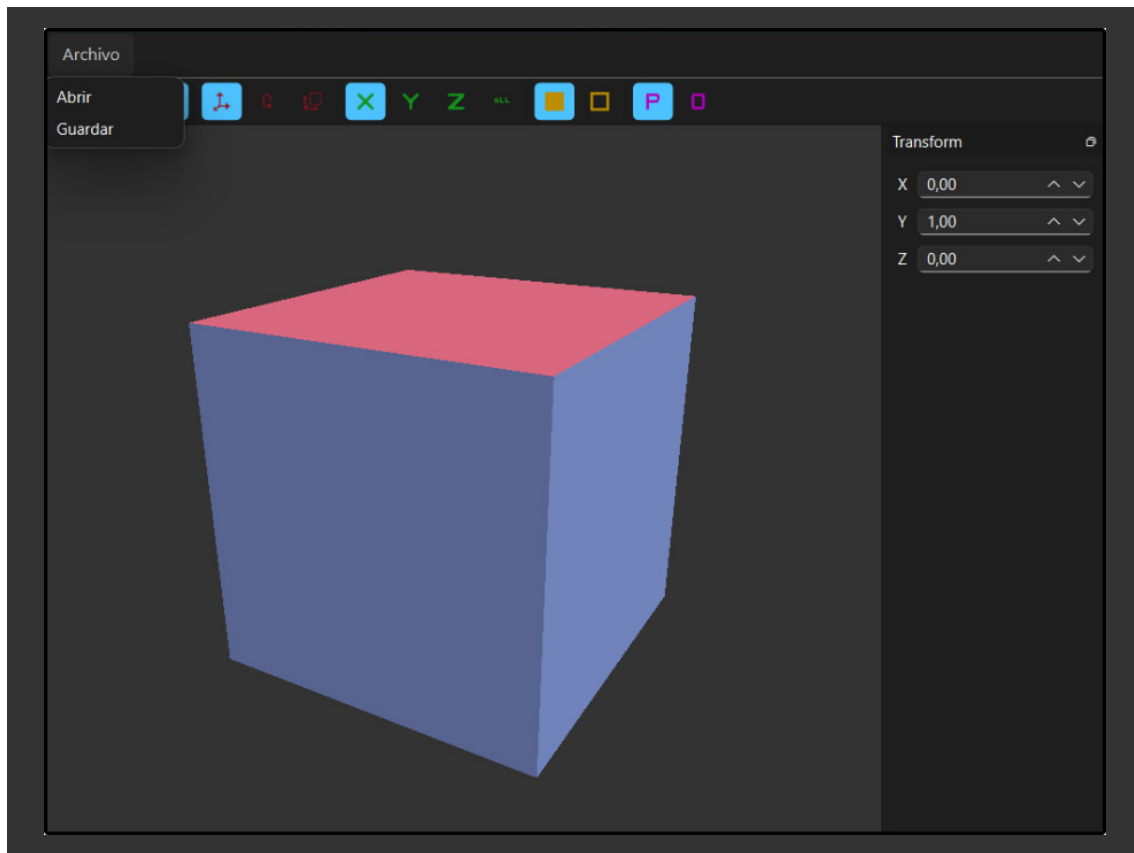


Figura 3.2: Interfaz de usuario de la aplicación.

y mediante la rueda del ratón se acerca o aleja la cámara (zoom). Adicionalmente, la tecla F recentra la cámara sobre el modelo (figura 3.3).

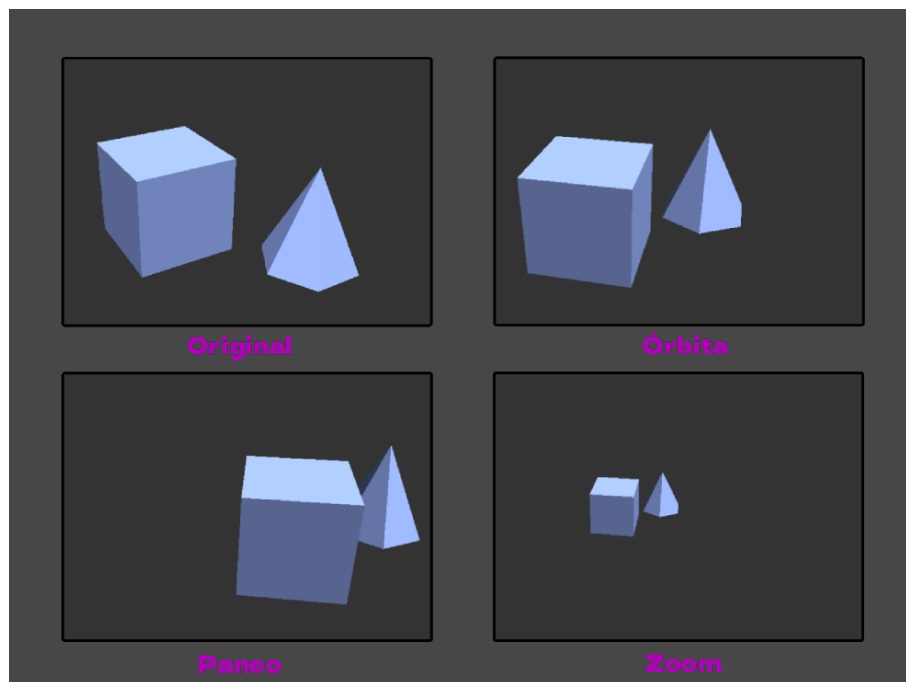


Figura 3.3: Controles de la cámara.

El editor ofrece dos tipos de proyección, seleccionables desde la barra de herramientas: perspectiva, en la que los objetos más alejados de la cámara aparecen de menor tamaño, y ortogonal, en la que dicho efecto no se produce.

Asimismo, el modelo puede visualizarse en modo sólido, con su superficie completamente rellena, o en modo *wireframe*, mostrando únicamente las aristas de la malla (figura 3.4).

3.2.3. Selección de elementos

El usuario puede seleccionar tres tipos de elementos de la malla: vértices, aristas o polígonos, alternando entre ellos desde la barra de herramientas. Al hacer clic izquierdo sobre un elemento, este queda seleccionado; manteniendo pulsada la tecla **Shift** durante el clic, la selección se añade a la ya existente en lugar de sustituirla, permitiendo seleccionar varios elementos a la vez (figura 3.5).

3.2.4. Edición de la malla

Sobre los elementos seleccionados pueden aplicarse las siguientes operaciones:

- **Transformación:** traslación, rotación o escalado de la selección, arrastrando el ratón tras elegir el modo correspondiente en la barra de herramientas. La

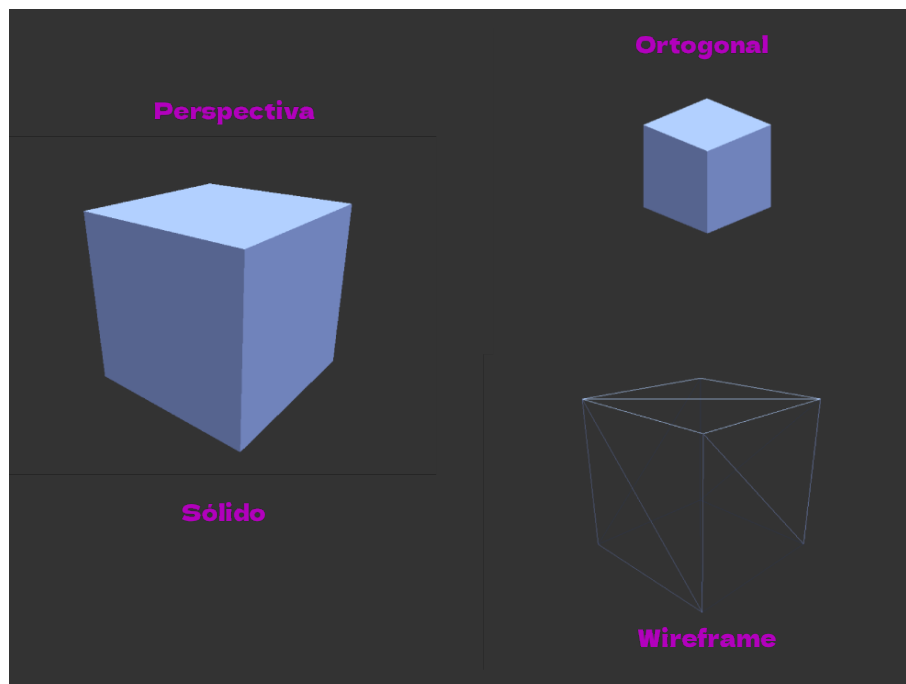


Figura 3.4: Comparación entre proyección perspectiva/ortogonal, y renderizado sólido/wireframe.

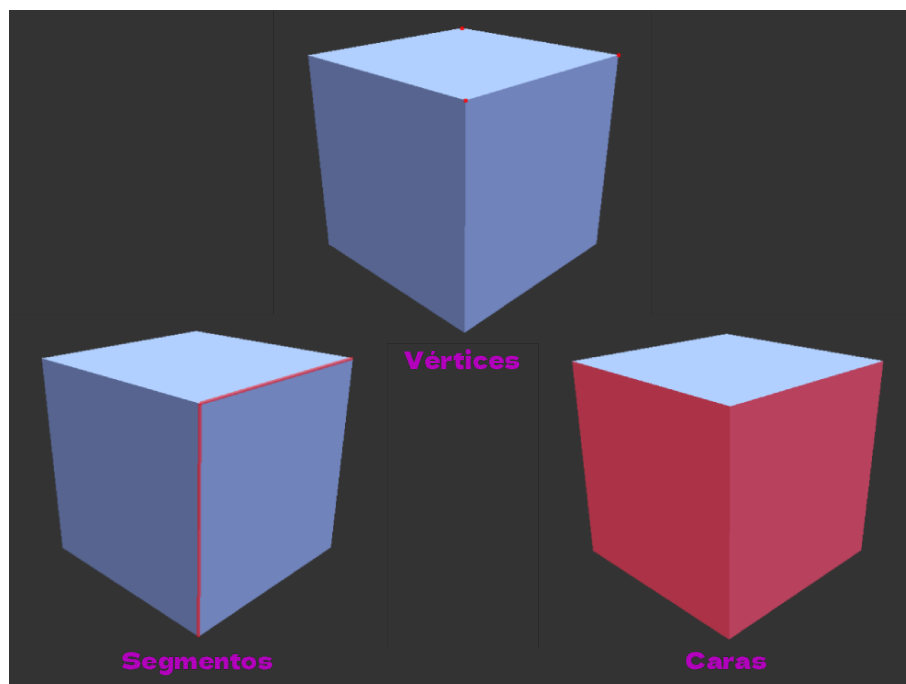


Figura 3.5: Selección de vértices, segmentos y polígonos

transformación puede restringirse a un único eje (X, Y o Z) o aplicarse libremente sobre los tres a la vez. Adicionalmente, la posición del elemento seleccionados (o posición media, en el caso de múltiples elementos) puede ajustarse numéricamente desde un panel lateral de la interfaz (figura 3.2) (figura 3.6).

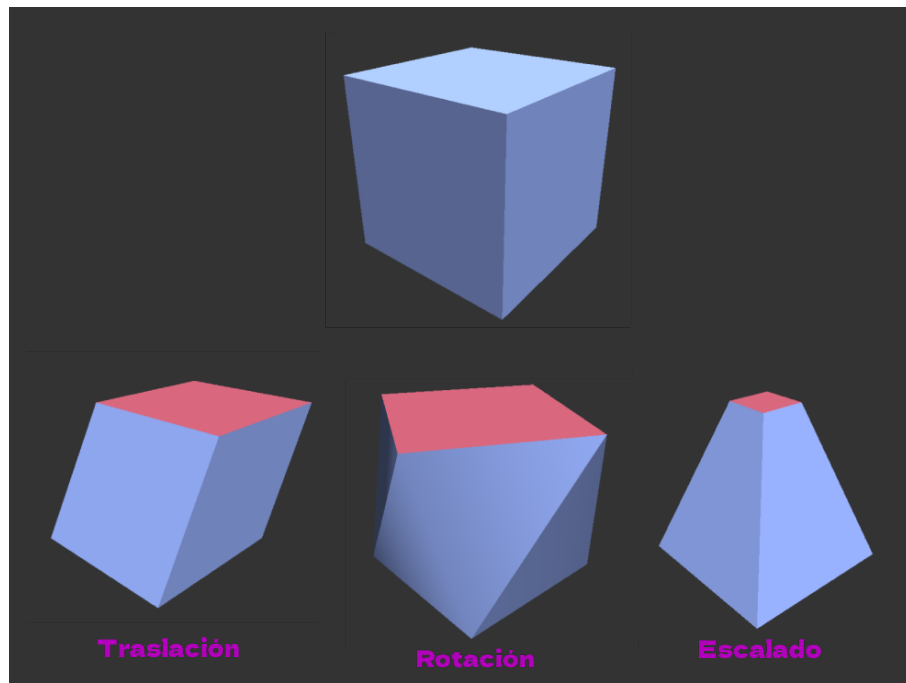


Figura 3.6: Tipos de transformaciones

- **Eliminación:** borra los elementos actualmente seleccionados de la malla (figura 3.7).
- **Extrusión:** genera nueva geometría a partir de los polígonos seleccionados, desplazándolos en la dirección de su normal y conectando los bordes resultantes con la superficie original (figura 3.7).

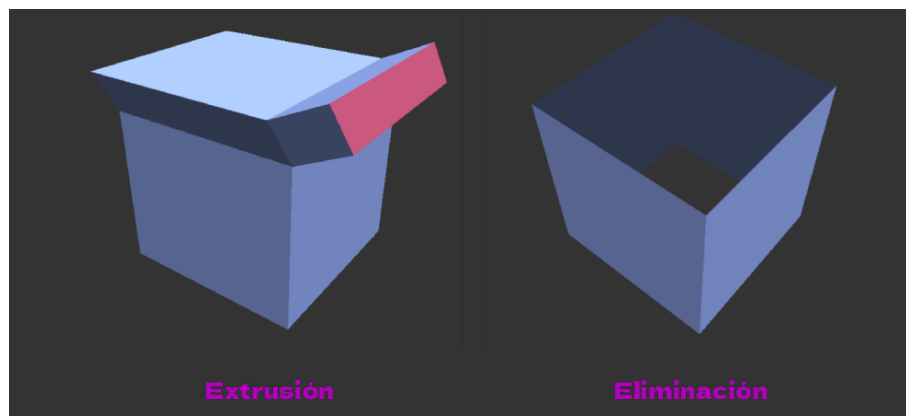


Figura 3.7: Extrusión y eliminación de caras seleccionadas.

Durante todo el proceso de edición, los elementos actualmente seleccionados se resaltan visualmente sobre la malla, permitiendo al usuario identificar en todo momento sobre qué está operando.

3.2.5. Atajos de teclado

La tabla 3.1 resume los atajos de teclado disponibles en el editor.

Tecla	Acción
1 / 2 / 3	Seleccionar vértices / aristas / polígonos
G / R / S	Modo de transformación: mover / rotar / escalar
X / Y / Z / A	Restringir transformación al eje X / Y / Z / todos
Q / W	Modo de renderizado: sólido / wireframe
P / O	Proyección: perspectiva / ortogonal
E	Extruir la selección poligonal
Supr	Eliminar la selección
Clic	Selección / deselección
Shift (+ clic)	Selección aditiva
F	Recentrar la cámara
Botón central + arrastre	Orbitar la cámara
Shift + botón central + arrastre	Desplazar la cámara (paneo)
Rueda del ratón	Acercar / alejar la cámara (zoom)

Tabla 3.1: Atajos de teclado del editor.

3.3. Detalles de implementación

A continuación se detalla el funcionamiento de los distintos componentes y sistemas del programa.

3.3.1. Sistema de renderizado

El primer requisito del editor consiste en disponer de un sistema capaz de representar modelos tridimensionales.

3.3.1.1. Creación de ventana y contexto con Qt

La biblioteca seleccionada para la creación de la ventana y el contexto de la aplicación fue Qt. Dentro de Qt se define un `QOpenGLWidget Canvas`, que gestiona el bucle principal de la aplicación a través del método `paintGL`. Este `Canvas` contiene al `Editor`, responsable de la lógica del programa, cuyo bucle principal es invocado desde el propio `Canvas` en cada frame.

Para poder invocar las funciones de OpenGL desde **Canvas**, es necesario obtener el contexto de OpenGL en tiempo de ejecución. Para ello, se recurre a (`QOpenGLContext::currentContext()`), a partir del cual se obtienen las direcciones de las funciones de OpenGL necesarias.

3.3.1.2. Shaders

Los shaders se almacenan en archivos independientes y se cargan durante la inicialización del sistema de renderizado. Estos programas están implementados en GLSL, el lenguaje de shaders de OpenGL, orientado a programas gráficos y que proporciona operaciones específicas entre vectores y matrices (de Vries, s.f.b, 2020).

La aplicación utiliza dos programas de shaders: uno de *iluminación*, empleado para representar la superficie de la malla, y uno de *selección*, utilizado para resaltar visualmente los elementos seleccionados por el usuario durante la edición. Cada programa está compuesto por un shader de vértices y un shader de fragmentos.

Shader de iluminación: El shader de vértices (mostrado en 1) recibe como atributos de entrada la posición y la normal de cada vértice. A partir de estos, calcula la posición del vértice en coordenadas de mundo, multiplicando su posición local por la matriz de modelo, y transforma su normal mediante la matriz normal (la traspuesta de la inversa de la matriz de modelo), necesaria para que la normal se transforme correctamente incluso en presencia de escalados no uniformes de la malla. Ambos valores se envían como variables de salida, que el rasterizador interpola automáticamente para cada fragmento situado dentro del triángulo antes de llegar al shader de fragmentos.

```
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;

uniform mat4 MVP;
uniform mat4 model;
uniform mat3 normalMatrix;

out vec3 FragPos;
out vec3 Normal;

void main()
{
    FragPos = vec3(model * vec4(aPos, 1.0));
    Normal = normalMatrix * aNormal;

    gl_Position = MVP * vec4(aPos, 1.0);
}
```

Listing 1: Shader de vértices del programa de iluminación.

El shader de fragmentos (listing 2) calcula la iluminación de cada fragmento a partir de la normal interpolada y de una única fuente de luz direccional. La dirección de dicha luz se recibe mediante el uniform `lightDir`, cuyo valor se fija directamente desde el código de la aplicación como un vector constante, sin que el usuario pueda modificarlo desde la interfaz.

El modelo de iluminación empleado puede entenderse como una instancia particular del modelo de Phong en la que el término especular es nula, quedando únicamente los términos ambiente y difuso. El término ambiente es una constante fija, y el término difuso se calcula según la ley de Lambert, esto es, el producto escalar entre la normal de la superficie y la dirección hacia la luz, limitado a valores no negativos para evitar iluminar caras opuestas a la fuente. El resultado se aplica directamente sobre el color base del objeto (`objectColor`). Al no definirse un color independiente para la luz, esta se comporta como una fuente de luz blanca.

Cabe destacar que, al calcularse la iluminación por fragmento a partir de la normal interpolada en lugar de en el vértice, como ocurre en *Gouraud shading*, la técnica empleada se corresponde con lo que se conoce como *Phong shading*. Este término no debe confundirse con el modelo de iluminación de Phong: el primero es una técnica de interpolación, mientras que el segundo es el modelo de iluminación en sí (ambiente, difusa y especular), del cual, en este caso, se emplea la variante con componente especular nulo.

```
out vec4 FragColor;

in vec3 FragPos;
in vec3 Normal;

uniform vec3 lightDir;
uniform vec3 objectColor;

void main()
{
    vec3 norm = normalize(Normal);
    vec3 light = normalize(-lightDir);

    float diff = max(dot(norm, light), 0.0);

    float ambient = 0.2;

    vec3 color = (ambient + diff) * objectColor;

    FragColor = vec4(color, 1.0);
}
```

Listing 2: Shader de fragmentos del programa de iluminación.

Shader de selección: El shader de selección tiene como propósito resaltar visualmente los elementos de la malla que el usuario tiene actualmente seleccionados. A diferencia del shader de iluminación (párrafo 3.3.1.2), no recibe la normal de los vértices como atributo de entrada, únicamente su posición, ya que no necesita calcular ningún tipo de sombreado. En consecuencia, su shader de vértices (listing 3) se limita a transformar dicha posición.

```
layout(location = 0) in vec3 vp;

uniform mat4 MVP;

void main()
{
    gl_Position = MVP * vec4(vp, 1.0);
}
```

Listing 3: Shader de vértices del programa de selección.

El shader de fragmentos asigna directamente un color rojo semitransparente a cada fragmento, sin depender de ninguna variable de entrada (listing 4).

```
out vec4 FragColor;

void main()
{
    FragColor = vec4(1.0, 0.0, 0.0, 0.5);
}
```

Listing 4: Shader de fragmentos del programa de selección.

La selección se resuelve previamente en la CPU: la clase **Selector** mantiene los índices de los vértices, aristas o polígonos actualmente seleccionados, a partir de los cuales se construye, en cada frame, un vector con las posiciones de todos los vértices contenidos en estos elementos. Para los polígonos, dado que pueden tener más de tres vértices, se recurre a la misma triangulación en abanico empleada durante su selección (sección 3.3.3.3), añadiendo al vector los tres vértices de cada triángulo resultante.

Este vector se envía a la clase **SelectionRenderer**, que mantiene un único VAO y VBO compartido entre los tres tipos de elemento seleccionable. El *buffer* se redimensiona en cada frame según el número de vértices a representar, mediante una llamada a `glBufferData`. Puesto que el modo de selección es excluyente entre vértices, aristas y polígonos, en un frame dado únicamente se dibuja un tipo de primitiva, lo que permite emitir toda la selección en una única llamada a `glDrawArrays`, con la primitiva de OpenGL correspondiente al tipo de elemento seleccionado (`GL_POINTS`, `GL_LINES` o `GL_TRIANGLES`).

3.3.1.3. Buffers

Las mallas del editor mantienen su propio VBO, EBO y VAO, gestionados por la clase `Mesh`, asumiendo simultáneamente el papel de modelo de datos de la malla y el de propietaria de los recursos de OpenGL necesarios para su renderizado, sin que exista una separación entre una representación puramente lógica de la malla y una representación orientada a su visualización.

Los atributos de vértice se almacenan de forma entrelazada en un único VBO: cada elemento consecutivo del buffer es una estructura `Vertex` completa, que agrupa la posición y la normal de un mismo vértice de renderizado. De este modo, el contenido del buffer en memoria sigue el patrón `[posición0, normal0, posición1, normal1, ...]`.

Para que la GPU pueda interpretar correctamente este entrelazado, el VAO configura dos atributos sobre el mismo buffer, cada uno con un *stride* igual al tamaño completo de la estructura `Vertex`, de modo que al leer un atributo la GPU sepa cuántos bytes debe avanzar para llegar al mismo atributo del siguiente vértice, saltando los datos intermedios que no le corresponden, y un *offset* distinto para cada uno, que indica en qué posición dentro de dicha estructura comienza el atributo en cuestión: 0 para la posición, y el desplazamiento de `Normal` dentro de `Vertex` (obtenido mediante `offsetof`) para la normal (listing 5).

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), nullptr);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex),
→ (void*)offsetof(Vertex, Normal));
```

Listing 5: Configuración de atributos en el VAO.

El EBO almacena los índices de cada polígono, de forma que la malla se renderiza como una malla indexada: los vértices compartidos entre varios triángulos de un mismo polígono se almacenan una única vez en el VBO y se referencian repetidamente desde el EBO.

Dado que la malla puede modificarse durante la edición, ambos buffers se crean con `GL_DYNAMIC_DRAW`, y se actualizan explícitamente cada vez que cambia la topología de la malla, volviendo a enviar su contenido completo a la GPU.

Cabe destacar que el resaltado visual de los elementos seleccionados (sección ??) no reutiliza estos buffers: se renderiza mediante un componente independiente, encargado de generar y enviar a la GPU la geometría auxiliar correspondiente a los vértices, aristas o polígonos seleccionados en cada frame.

3.3.2. Representación de la malla

Para poder editar la malla mostrada en la ventana previamente creada, se define una representación interna de esta en el programa.

A pesar de aprovechar parcialmente la estructura de malla proporcionada por

`tinyobjloader`, esta no ofrece dos elementos necesarios para la edición del modelo: una representación de las aristas de la malla, y una agrupación de los vértices que comparten una misma posición geométrica. Por este motivo se desarrolla una representación interna propia, descrita en los siguientes apartados.

Durante la carga del modelo es necesario transformar la representación proporcionada por `tinyobjloader` en esta representación interna. Este proceso resuelve dos necesidades distintas: por un lado, evitar la duplicación de vértices de renderizado idénticos, esto es, con la misma posición y la misma normal, y por otro, mantener agrupados aquellos vértices de renderizado que comparten una misma posición geométrica pero que, al pertenecer a caras con distinta orientación, poseen normales distintas y por tanto no pueden fusionarse en un único vértice de renderizado. El primer proceso reduce el número de vértices almacenados, mientras que el segundo no elimina ningún vértice, sino que mantiene una referencia entre aquellos que, aun siendo distintos a nivel de renderizado, representan el mismo vértice lógico (sección 3.3.2.1).

Para la deduplicación de vértices idénticos, se utiliza un valor *hash* de cada vértice como clave para localizar posibles coincidencias dentro de un `unordered_map`. Dicho *hash* se calcula combinando mediante XOR el *hash* individual de cada una de las componentes del vértice (posición y normal). Una vez localizada una posible coincidencia por *hash*, se comparan los valores reales de posición y normal para confirmar que ambos vértices son efectivamente equivalentes.

Sin esta estructura, comparar cada nuevo vértice contra todos los ya almacenados tendría un coste cuadrático, $O(n^2)$, para n vértices. Empleando el mapa de *hash*, cada búsqueda se resuelve en tiempo constante, reduciendo el coste total esperado del proceso a $O(n)$. Este coste depende de una distribución uniforme de la función *hash* entre los vértices insertados. En el peor caso, si un número elevado de vértices distintos colisionase por tener una misma clave del mapa, el coste de cada búsqueda podría degradar hasta $O(n^2)$, siendo n el número de vértices que comparten dicha clave, ya que deben compararse individualmente para confirmar si alguno constituye una coincidencia real.

Para la agrupación por posición geométrica, se emplea directamente la posición de cada vértice como clave de un segundo mapa (`positionToGroup`), que asocia cada posición con su grupo correspondiente. Dado que `glm::vec3` no dispone de una función de *hash* en la biblioteca estándar, se define una función *hash* propia para este tipo. De este modo, dos vértices de renderizado se agrupan bajo el mismo vértice lógico solo si comparten exactamente la misma posición, con independencia de si el archivo OBJ original les asignó índices de posición distintos.

Cabe señalar una limitación de este proceso de comparación de posiciones, que al igual que la deduplicación descrita anteriormente, es una comparación exacta de valores en punto flotante: dos posiciones que fuesen geoméricamente equivalentes, pero que difiriesen en el redondeo de sus últimos decimales, no se reconocerían como el mismo vértice lógico.

3.3.2.1. Vértices

Cada vértice de renderizado almacena, además de su posición, una normal utilizada durante el proceso de iluminación del modelo. Si el archivo OBJ cargado incluye normales, estas se copian directamente durante la carga. En caso contrario, tras una operación de edición que modifique la geometría, se recalculan por acumulación de normales de cara mediante el método de Newell (?): para cada triángulo se calcula su normal mediante el producto vectorial de dos de sus aristas, dicha normal se acumula sobre cada uno de los tres vértices que lo componen y, finalmente, cada normal acumulada se normaliza.

Para el renderizado mediante OpenGL, la geometría se representa mediante primitivas triangulares, formadas por grupos de tres vértices. Esta es la unidad más simple de una malla. Sin embargo, un mismo punto del espacio puede requerir varios vértices de renderizado distintos. Un cubo, por ejemplo, posee geométricamente ocho posiciones de vértice, pero cada una de ellas es compartida por tres caras con orientaciones distintas, y por tanto con normales distintas. Dado que un vértice de renderizado solo puede almacenar una única normal, cada posición geométrica del cubo debe representarse mediante un vértice de renderizado independiente por cada cara a la que pertenece, resultando en veinticuatro vértices de renderizado en total (cuatro por cada una de las seis caras).

Aún así, a nivel de usuario, resulta más natural razonar sobre las ocho posiciones geométricas del cubo que sobre sus veinticuatro vértices de renderizado: al mover uno de los vértices de una esquina, por ejemplo, se espera que se desplacen conjuntamente los tres vértices de renderizado asociados a dicha esquina. Por este motivo, el programa mantiene dos estructuras adicionales: un listado de grupos de vértices (`std::vector<std::vector<unsigned int>> logicGroups`), donde cada grupo contiene los índices de todos los vértices de renderizado que comparten una misma posición geométrica, y su mapeo inverso (`std::vector<unsigned int> vertexToGroup`), donde la posición i de este vector almacena directamente el grupo al que pertenece el vértice de renderizado i .

En lo sucesivo, nos referiremos a los vértices tal como los interpreta OpenGL (los veinticuatro del cubo) como *vértices de renderizado*, y a las posiciones geométricas únicas que agrupan (los ocho del cubo) como *vértices lógicos*.

3.3.2.2. Aristas

Se considera una arista como un par de vértices conectados, representado mediante el struct `Edge`, que almacena los índices de renderizado de sus dos vértices. Una vez cargada la malla, se almacenan todas las aristas relevantes en un vector, con el fin de poder operar sobre ellas posteriormente sin necesidad de reconstruirlas a partir de los polígonos en cada acceso. Como nota, el motivo de almacenar los índices de renderizado y no los de su vértice lógico, es el acceso inmediato sin la necesidad de una indirección adicional que supondría acceder a la posición de esta.

Para generarlas, se recorren los pares de vértices adyacentes de cada polígono que conforma el modelo y se añaden ordenadamente. Con el objetivo de evitar duplicar

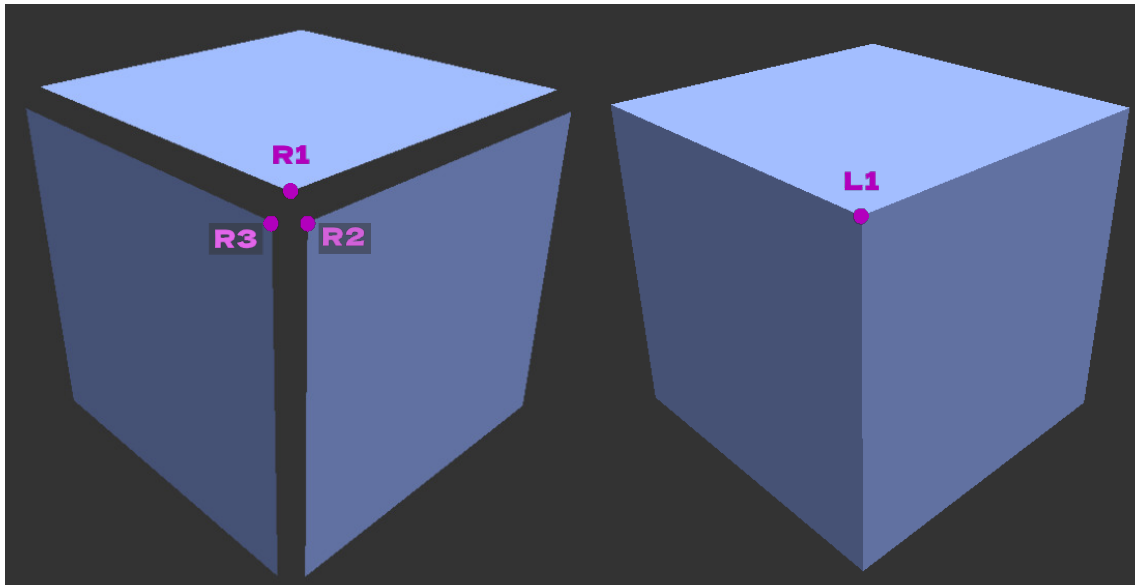


Figura 3.8: Tres vértices de renderizado se manipulan con uno lógico.

una arista compartida por dos polígonos adyacentes, se comprueba si este ya ha sido añadido a partir del vértice lógico al que pertenece cada vértice de renderizado, y no de su índice de renderizado directo, ya que dos caras distintas pueden compartir arista utilizando vértices de renderizado distintos entre sí (por poseer normales diferentes). Además, cada par de vértices lógicos se ordena numéricamente, colocando el índice de grupo menor en primer lugar, de manera que una arista $V_1 - V_2$ y su equivalente $V_2 - V_1$ se consideran la misma arista, independientemente del orden en que aparezcan al recorrer los polígonos.

Este proceso de deduplicación tiene un coste de $O(P \log P)$, siendo P el número total de pares de vértices adyacentes recorridos, ya que se emplea un conjunto ordenado (`std::set`) para comprobar la existencia previa de cada arista. Una mejora a esto podría ser la implementación en su lugar de una tabla de *hash*, reduciendo su coste a $O(P)$ en promedio.

Este proceso tiene lugar tras la carga completa del modelo, de manera que los vértices lógicos de `vertexToGroup` están completamente contruidos, y la generación de polígonos también ha concluido, puesto que las aristas se construyen a partir de la representación interna de los polígonos.

Almacenar las aristas de forma explícita, en lugar de calcularlas a partir de los polígonos cada vez que se necesitan, resulta especialmente relevante para dos operaciones: la selección de aristas, que itera directamente sobre esta lista en cada iteración del bucle principal de la aplicación, y el resaltado visual de las aristas seleccionadas durante la edición.

3.3.2.3. Polígonos

Denominamos polígono a la cara lógica de la malla, esto es, la entidad geométrica con la que interactúa el usuario durante la edición. Cada polígono se representa

mediante la estructura `Polygon`, que almacena una lista ordenada de índices a los vértices de renderizado que delimitan su contorno, al igual que con las aristas, con el objetivo de reducir indirecciones en algunas de sus funciones.

Para el renderizado mediante OpenGL, sin embargo, la geometría debe descomponerse en primitivas triangulares, por lo que cada polígono puede representarse mediante uno o varios triángulos. Durante la carga de la malla, `tinyobjloader` permite configurar si las caras del modelo se cargan ya trianguladas o si se conserva su estructura original. Dado que, a nivel de usuario, la edición de una malla resulta más intuitiva manteniendo las caras completas en lugar de trabajar directamente sobre triángulos individuales, se opta por este segundo modo, conservando la estructura de cada polígono en la representación interna de la malla.

Los polígonos se almacenan en un vector y se identifican mediante su posición dentro de esta estructura, lo que permite acceder directamente al polígono correspondiente durante el proceso de selección.

3.3.3. Selección

El objetivo de este módulo es permitir al usuario seleccionar los distintos elementos lógicos que conforman la malla, ya sean vértices, aristas o polígonos, para posteriormente realizar transformaciones sobre ellos.

Para indicar qué tipo de elemento desea seleccionar, se muestran botones en la interfaz, uno para cada tipo de elemento: selección de vértices, aristas y polígonos. El procedimiento general para detectar qué elemento se está seleccionando consiste en la proyección de los vértices tridimensionales sobre la pantalla, para obtener tanto su posición en dos dimensiones como su profundidad respecto a la cámara. Tras ello, en función del modo de selección, se aplica un algoritmo específico para detectar el elemento.

En consecuencia, el programa calcula y almacena la posición bidimensional y la profundidad de cada vértice de la malla en cada iteración del bucle principal de la aplicación, para poder acceder a estos valores rápidamente durante el resto del proceso de selección.

3.3.3.1. Selección de vértices

Existen dos formas principales de determinar cuál es el vértice que el usuario intenta seleccionar: por un lado, se puede realizar *ray casting* 2.3.5 y calcular la distancia entre este rayo y los vértices que componen la malla, para elegir aquél cuya proximidad sea mayor. La alternativa consiste en proyectar los vértices del espacio tridimensional sobre la pantalla, donde se puede posteriormente calcular la distancia euclidiana respecto a la posición del ratón con `glm::distance`.

En este programa se opta por la segunda opción. El motivo de esta elección es el mejor rendimiento obtenido al operar en dos dimensiones frente a tres.

Dicha proyección sobre la pantalla se calcula una única vez por cada iteración

del bucle principal de la aplicación, y se reutiliza por igual en los tres modos de selección (vértices, aristas y polígonos), evitando así proyectar la misma geometría por separado para cada uno. Además, la proyección se realiza únicamente sobre los vértices lógicos de la malla y no sobre sus vértices de renderizado: dado que todos los vértices de renderizado de un mismo grupo comparten exactamente la misma posición (sección 3.3.2.1), basta con proyectar un único representante por grupo (en nuestro caso, el primer vértice de renderizado de cada uno), evitando así proyecciones redundantes. Con ello, el coste de la proyección es de $O(n)$, siendo n el número de vértices lógicos de la malla.

Un detalle de interés es que, al proyectar los vértices sobre la pantalla, además de su posición en dos dimensiones se almacena la profundidad a la que se encuentran respecto a la cámara. Si tan solo se midiera la distancia mínima del ratón a cada vértice proyectado, los resultados no serían acertados: dos vértices superpuestos en pantalla, pero situados a distinta profundidad en la escena, resultarían indistinguibles para el criterio de selección. Por ello, el vértice candidato se determina combinando ambos valores: se considera mejor candidato aquél cuya distancia sea claramente menor que la del mejor candidato encontrado hasta el momento. Si la diferencia de distancias entre ambos es despreciable (por debajo de un pequeño margen de tolerancia `distanceEpsilon`), se prioriza en su lugar el de menor profundidad. De esta manera se resuelven correctamente los casos de vértices próximos entre sí en pantalla, seleccionando el que visualmente se encuentre más cerca de la cámara.

3.3.3.2. Selección de aristas

El primer paso consiste en recorrer todas las aristas de la malla y, para cada una, comprobar si sus dos vértices se encuentran dentro del volumen de vista de la cámara. Dicho de otra forma, si son visibles para la cámara. En caso contrario, la arista se descarta. Debe notarse que esto puede resultar en una arista parcialmente visible quedando descartada, en el caso de que solamente uno de sus vértices sea visible.

A continuación se calcula el punto del segmento proyectado en pantalla más cercano al ratón. Para ello se proyecta la posición del ratón sobre la recta que definen los dos vértices de la arista, obteniendo un parámetro t que indica en qué punto del segmento se sitúa dicha proyección: $t = 0$ corresponde al primer vértice y $t = 1$ al segundo. Este parámetro se limita al intervalo $[0, 1]$, de modo que el punto más cercano se calcula respecto al segmento y no a la recta infinita que lo contiene. Con ello se obtiene el punto del segmento más próximo al ratón, y la distancia euclidiana entre ambos (listing 6).

Al igual que con los vértices, la selección de la arista depende de la profundidad a la que se encuentran sus vértices respecto a la cámara. Esta profundidad se interpola linealmente a lo largo de la arista utilizando el mismo parámetro t calculado anteriormente, obteniendo así una estimación de la profundidad del punto de contacto entre el ratón y la arista, que se evalúa junto con la distancia al ratón para determinar el mejor candidato.

```

const ProjectedVertex& pa = ...;
const ProjectedVertex& pb = ...;

if (!pa.visible || !pb.visible)
    continue;

glm::vec2 ab = pb.screenPosition - pa.screenPosition;

// Calculo de longitud del vector ab(hacerlo asi evita usar raices
→ cuadradas)
float len = glm::dot(ab, ab);

// Ignoramos si el segmento es muy pequeño
if (len < 0.00001f)
    continue;

// Determinamos la posicion del raton t sobre el segmento, que resulta en
→ valores entre 0(sobre a) y 1(sobre b)
float t = glm::dot(mouse - pa.screenPosition, ab) / len;
t = glm::clamp(t, 0.0f, 1.0f); // Mantenemos valor entre 0 y 1

// Punto del segmento mas cercano al raton
glm::vec2 closest = pa.screenPosition + t * ab;

float dist = glm::distance(mouse, closest);
float depth = pa.depth * (1.0f - t) + pb.depth * t;

// Determinamos si es el mejor candidato (selectedEdge)
if (isBetterCandidate(dist, depth, minScreenDistance, minDepth)) {
    selectedEdge = i;
}

```

Listing 6: Determinación de mejor candidato entre aristas.

Finalmente, un detalle adicional es que solo se considera una arista como candidata válida si su distancia al ratón se encuentra por debajo de un umbral mínimo en píxeles, evitando selecciones accidentales de aristas alejadas del cursor.

3.3.3.3. Selección de polígonos

Para seleccionar polígonos, se comienza recorriendo todos ellos y descartando aquellos que posean algún vértice fuera del volumen de vista de la cámara.

Para los polígonos restantes, se comprueba si el ratón se encuentra dentro de alguno de los triángulos en los que se descompone cada polígono. Esta descomposición se realiza mediante triangulación en abanico (figura 3.9): se fija el primer vértice del polígono como vértice común, y se forman triángulos tomando, de dos en dos, cada par de vértices consecutivos restantes junto con dicho vértice común.

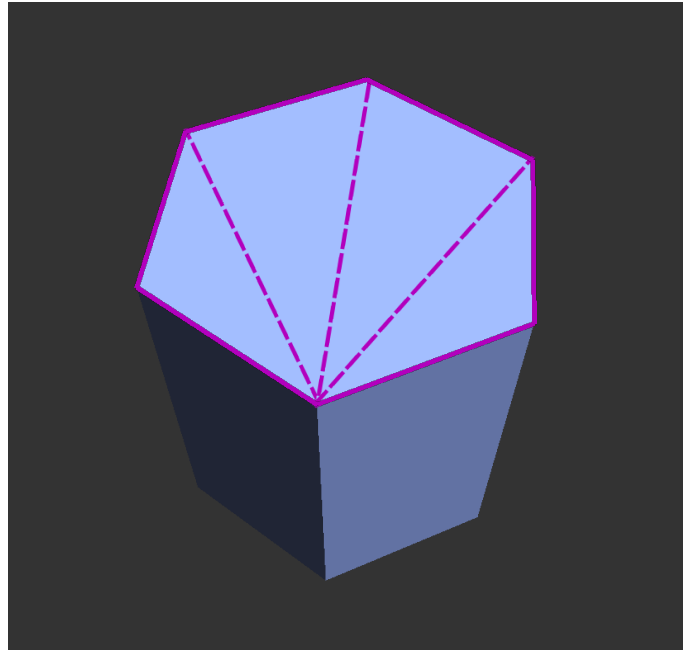


Figura 3.9: Triangulación de polígono en abanico.

Para determinar si el ratón se encuentra dentro de cada uno de estos triángulos, se emplea el cálculo de coordenadas baricéntricas (código 7). Si alguno de los triángulos en los que se divide el polígono contiene la posición del ratón, se calcula la profundidad que tiene respecto a la cámara, y si es el polígono que se encuentra a menor profundidad de entre todos los candidatos, queda marcado como mejor candidato.

3.3.4. Cámara

Manipular un elemento tridimensional requiere que el usuario lo pueda observar desde múltiples ángulos. Con este fin se desarrolla una cámara que ofrece al usuario diversas opciones para manejarla.

Tal como se describe en la sección 2.3.2, la cámara interviene en el pipeline de renderizado a través de la matriz de vista. Para su funcionamiento, se definen las variables de posición de la cámara, el objetivo hacia el que mira, y la dirección “arriba” del mundo (en este caso, el eje $(0, 1, 0)$, por convención general). Estas variables permiten posteriormente realizar las distintas operaciones sobre cómo la cámara observa las mallas del mundo tridimensional. Además, se mantiene una variable para indicar el tipo de proyección que usa la cámara, perspectiva u ortogonal, que también afecta al funcionamiento del zoom, como se describe más adelante.

Con todo esto se desarrollan las tres operaciones principales con las que se puede manipular la cámara:

Órbita: la cámara gira alrededor del objetivo indicado, mediante el uso del botón central del ratón. Para determinar la nueva posición de la cámara se toma el desplazamiento del ratón en pantalla en los ejes X e Y , y en función de este

```

bool Selector::pointInTriangle(const glm::vec2& p, const glm::vec2& a,
                               const glm::vec2& b, const glm::vec2& c,
                               float& u, float& v) {

    glm::vec2 v0 = c - a;
    glm::vec2 v1 = b - a;
    glm::vec2 v2 = p - a;

    float dot00 = glm::dot(v0, v0);
    float dot01 = glm::dot(v0, v1);
    float dot02 = glm::dot(v0, v2);
    float dot11 = glm::dot(v1, v1);
    float dot12 = glm::dot(v1, v2);

    float invDenom = 1.0f / (dot00 * dot11 - dot01 * dot01);
    u = (dot11 * dot02 - dot01 * dot12) * invDenom;
    v = (dot00 * dot12 - dot01 * dot02) * invDenom;

    return (u >= 0.0f) && (v >= 0.0f) && (u + v <= 1.0f);
}

```

Listing 7: Comprobación de punto dentro de triángulo mediante cálculo de coordenadas baricéntricas.

se modifican el *pitch* y el *yaw* de la cámara (los ángulos de inclinación vertical y de giro horizontal respecto al objetivo). Para evitar saltos bruscos al alcanzar una orientación paralela al eje vertical, se establecen límites máximos para el valor del *pitch*. A continuación se calcula la nueva dirección de la cámara, que se normaliza, y junto con la posición del objetivo y la distancia a la que se encuentra la cámara de este, se obtiene la nueva posición de la cámara.

Paneo: permite desplazar la vista en el plano actual de la cámara, tanto horizontal como verticalmente. Se usa nuevamente el desplazamiento del cursor en pantalla para determinar cuánto deben desplazarse la cámara y su objetivo, cambiando así el punto alrededor del cual orbita la cámara.

Zoom: varía su funcionamiento en función del tipo de proyección de la cámara. En perspectiva, se incrementa o decrementa la distancia entre la cámara y el objetivo, lo cual sí produce un efecto de acercamiento visual, ya que en este tipo de proyección las líneas de proyección convergen hacia la cámara, y por tanto el tamaño aparente de un objeto en pantalla depende de su distancia a esta. En la proyección ortogonal, en cambio, las líneas de proyección son paralelas entre sí en lugar de converger hacia la cámara, por lo que el tamaño aparente de un objeto no varía con su distancia a la cámara, y acercarla o alejarla no produciría ningún efecto visual de zoom.

Por este motivo, en proyección ortogonal el zoom actúa en su lugar directamente sobre el volumen de vista: la variable `orthogonalZoom` determina la mitad de la altura de dicho volumen, a partir de la cual se calcula su mitad de anchura aplicando la relación de aspecto de la ventana, y ambos valores se emplean para construir la matriz de proyección ortogonal. Reducir `orthogonalZoom` estrecha el volumen de

vista, haciendo que el modelo ocupe una porción mayor de la pantalla, y aumentarlo produce el efecto contrario.

Un detalle común a las tres operaciones de la cámara es que, antes de aplicarlas, el desplazamiento del ratón se multiplica por la distancia actual de la cámara al objetivo: de esta manera, si la cámara se encuentra ya próxima a este, el movimiento resultante será menor, y viceversa, permitiendo al usuario un control más preciso. La única excepción es el zoom en proyección ortogonal, que modifica `orthogonalZoom` en una cantidad fija por cada evento de *scroll*, sin multiplicarla por ninguna distancia: al no existir, en este modo, una distancia entre cámara y objetivo que determine el tamaño aparente del modelo (según se ha explicado), no hay ninguna magnitud equivalente por la que escalar el movimiento, y su sensibilidad percibida no varía en función de lo alejada o cercana se encuentre la cámara del objetivo.

Finalmente, se debe tener en cuenta el espacio de recorte de la cámara: una vez obtenida la proyección de vista, se descarta aquello que se encuentre demasiado cerca o demasiado lejos de la cámara. Para ello, la cámara mantiene dos variables de recorte: una que indica la distancia mínima por debajo de la cual un objeto deja de renderizarse por estar demasiado cerca, establecida en 0.1 unidades, y otra para la distancia máxima, establecida en 100 unidades.

3.3.5. Operaciones de manipulación

El proyecto ofrece diversas operaciones realizables sobre el elemento actualmente seleccionado.

3.3.5.1. Transformación

La operación de transformación permite aplicar traslación, rotación o escalado sobre los elementos seleccionados. Se inicia al pulsar el botón del ratón sobre un elemento ya seleccionado, se actualiza en cada iteración del bucle principal mientras dicho botón permanece pulsado, y finaliza al soltarlo.

Al iniciarse, se establece el pivote de la transformación como la media aritmética de las posiciones de los vértices seleccionados, de modo que el resultado de rotar o escalar la selección se perciba centrado en torno a ella. A continuación, se almacenan las posiciones originales de todos los vértices que se verán afectados, y se calcula un plano de arrastre, definido por dicho pivote y la dirección desde la cámara hacia este.

Este plano no interviene por igual en las tres operaciones. En la traslación, se emplea para convertir el movimiento bidimensional del ratón en un desplazamiento tridimensional: en cada iteración se calcula el punto en el que el rayo proyectado desde la posición actual del ratón intersecta con dicho plano, y la diferencia respecto al punto de intersección inicial determina la magnitud de la traslación. La rotación y el escalado, en cambio, se basan directamente en el desplazamiento horizontal del ratón en pantalla, desde la posición en la que se inició la transformación. Únicamente cuando la rotación se realiza sin un eje fijo se reutiliza la normal de este plano,

empleándola en ese caso como eje de rotación, orientado de la cámara hacia el pivote, en lugar de como plano de intersección.

En el caso de la rotación, el ángulo se obtiene multiplicando el desplazamiento horizontal por un factor de sensibilidad. En el caso del escalado se sigue un proceso análogo, con la diferencia de que, para un desplazamiento nulo del ratón, el factor de escala debe mantenerse en 1, evitando así que los elementos seleccionados colapsen sobre sí mismos en el instante en que se inicia la transformación.

En los tres casos, la magnitud obtenida se aplica sobre una matriz identidad mediante las funciones correspondientes de `glm` (`translate`, `rotate` o `scale`), obteniendo así la matriz de transformación resultante. Dicha matriz se aplica a cada vértice seleccionado tomando como referencia su posición original respecto al pivote, y no su posición actual, evitando así una acumulación de error de redondeo si la transformación se actualiza durante muchas iteraciones consecutivas del bucle principal.

Además de mediante arrastre del ratón, la posición de la selección puede ajustarse numéricamente desde un panel de la interfaz (sección 3.1.1), que permite fijar directamente cada coordenada del pivote. A diferencia del arrastre, este mecanismo calcula directamente el desplazamiento necesario para que el pivote alcance el valor introducido, y aplica dicho desplazamiento como una traslación. Los dos tipos de transformación permanecen sincronizados, con una modificación del panel de la interfaz transformando el elemento seleccionado, y una alteración de posición de un elemento notificando a la interfaz de la nueva posición del pivote del elemento seleccionado.

Cabe señalar que el cálculo de la transformación se realiza íntegramente en la CPU: se recorren los vértices seleccionados en un bucle, y al finalizar se recalculan las normales de toda la malla y se reenvía el contenido completo del VBO a la GPU, en lugar de únicamente los vértices modificados (sección 3.3.1.3). Para mallas de gran tamaño, esto implica transferir a la GPU información no modificada en cada iteración del arrastre. Por tanto, una posible mejora sería trasladar el cálculo de la transformación a la propia GPU.

3.3.5.2. Eliminación de elementos

Otra manipulación disponible sobre la malla es la eliminación de elementos, cuyo comportamiento varía según el tipo de elemento seleccionado.

Si la selección actual son polígonos, estos se eliminan directamente de la lista de polígonos de la malla. Si la selección son aristas, para cada arista seleccionada se recorren todos los polígonos de la malla comprobando si alguna de sus aristas coincide con ella. Todo polígono que contenga la arista seleccionada queda marcado para su eliminación. Si la selección son vértices, se eliminan todos los polígonos que hagan uso de alguno de los vértices seleccionados. En los tres casos, el resultado del proceso es siempre el mismo: un conjunto de polígonos marcados para su eliminación, sobre el que se aplica a continuación un procedimiento común.

Dicho procedimiento no elimina explícitamente ninguna arista ni ningún vértice

de forma individual. Las aristas, al no almacenarse de forma incremental sino reconstruirse por completo a partir de los polígonos supervivientes (sección 3.3.2.2), dejan automáticamente de incluir cualquier arista que perteneciera únicamente a los polígonos eliminados, sin necesidad de localizarla y eliminarla de forma independiente. Los vértices, por su parte, pueden quedar sin ningún polígono que los referencie tras eliminar los polígonos marcados; por ello, se recorre la lista de vértices para identificar aquellos que han quedado sueltos, y se eliminan. Dado que esto desplaza los índices de los vértices restantes, se construye una tabla de remapeado que permite actualizar las referencias a vértices en los polígonos supervivientes, así como reconstruir los grupos de vértices lógicos, algunos de los cuales pueden haber desaparecido por completo si todos sus vértices asociados quedaron sueltos. Finalmente, se reconstruye la topología de la malla, lo cual regenera tanto las aristas como los índices de renderizado a partir de los polígonos ya actualizados.

Herramientas de edición de mallas como Blender tienden a ofrecer esta eliminación en cascada, aunque también ofrecen al usuario otras opciones explícitas sobre la geometría que desea conservar (Blender Foundation, 2026b).

3.3.5.3. Extrusión

Además de lo anterior, se pueden realizar operaciones de extrusión sobre la malla. Esta operación recibe un único parámetro, la distancia a extruir, y se invoca al pulsar una tecla determinada, aplicándose sobre la selección de polígonos actual con un valor de distancia fijo definido en el propio código. A diferencia de las operaciones de transformación descritas anteriormente, la extrusión no admite en su implementación actual un valor interactivo controlado por el arrastre del ratón.

Para realizarla, se comienza obteniendo la dirección media de las normales de los polígonos seleccionados, ya que esta será la dirección en la que se extruye la selección. A continuación, por cada vértice lógico presente en la selección se crea un nuevo vértice desplazado la distancia indicada en dicha dirección, quedando así diferenciados los vértices nuevos de los originales.

Antes de construir las caras laterales, se determina cuántos de los polígonos seleccionados hacen uso de cada arista. Esto permite distinguir las aristas internas de la selección, compartidas por dos caras seleccionadas adyacentes, de las aristas de borde, pertenecientes a una única cara seleccionada.

Para cada polígono seleccionado se genera un nuevo conjunto de vértices superiores, correspondientes a los vértices desplazados calculados anteriormente. Después, recorriendo cada arista del polígono original, se omite la creación de una cara lateral si dicha arista es interior a la selección. En caso contrario, se construye un nuevo polígono cuadrangular que une los dos vértices originales de la arista con sus correspondientes vértices desplazados, cerrando así el lateral de la extrusión. Finalmente, el polígono original se actualiza para que use directamente los nuevos vértices superiores, pasando a formar la cubierta de la extrusión, y se reconstruye la topología de la malla.

3.3.6. Ratón y teclado

La aplicación gestiona el input del usuario a través de dos vías independientes, teniendo en cuenta una peculiaridad del sistema de atajos de teclado de Qt.

Determinadas acciones de la interfaz (el modo de selección, el modo de transformación, el eje activo, el modo de renderizado y el tipo de proyección) se activan mediante teclas registradas como atajos de `QAction` en `MainWindow` con `QAction::setShortcut(...)`, cada una conectada directamente a un método del `Editor`. Qt resuelve estos atajos mediante su propio sistema de gestión de teclado a nivel de aplicación, el cual intercepta el evento correspondiente antes de que este llegue al *widget* de renderizado (`Canvas`). En consecuencia, ninguna de estas teclas resulta detectable a través del resto del sistema de *input* del programa.

El resto del *input* (resto de teclas del teclado, así como los botones y la rueda del ratón) se captura en los manejadores de eventos de `Canvas`, que reenvían cada evento a la clase `Input` a medida que Qt se los entrega. Esta clase se implementa mediante miembros y métodos estáticos, lo que permite consultar su estado desde cualquier punto del programa sin necesidad de una referencia.

Es `Editor` quien gestiona el comportamiento durante una iteración del bucle principal del programa para `Input`: al inicio de cada iteración invoca `Input::update()`, y al final, `Input::endFrame()`. Es `Editor` quien consulta, junto con `Camera`, dicho estado, y así determina el comportamiento del programa en cada iteración.

Para distinguir si una tecla se ha pulsado en el frame actual de si simplemente permanece pulsada, se mantiene una copia del estado del frame anterior (`previousPressedKeys`), actualizada al final de cada frame. La comparación entre ambos conjuntos permite ofrecer dos consultas distintas: `isKeyDown`, que indica si una tecla está pulsada en un instante dado, e `isKeyPressed`, que solo resulta cierta durante el frame exacto en que la tecla pasa a estar pulsada. El mismo mecanismo se aplica a los botones del ratón, permitiendo además detectar mediante `isMouseButtonUp` el instante en que se suelta un botón, empleado para finalizar una operación de transformación en curso.

El desplazamiento del ratón entre frames se calcula invirtiendo el signo de su componente vertical (código 8):

```
deltaX = mouseX - lastMouseX;  
deltaY = lastMouseY - mouseY;
```

Listing 8: Cálculo del desplazamiento del ratón entre frames.

Esto compensa que las coordenadas de pantalla de Qt crecen hacia abajo, mientras que un desplazamiento hacia arriba del ratón se espera como positivo en el resto del programa. Resolver esta inversión una única vez evita tener que repetirla en cada usuario del desplazamiento del ratón, como la órbita o el paneo de la cámara (sección 3.3.4).

Por último, a diferencia del resto del estado de input, el desplazamiento acumu-

lado de la rueda del ratón se reinicia al final de cada frame, ya que solo tiene sentido como una medida de cuánto se ha desplazado la rueda durante ese frame concreto: de no reiniciarse, el último valor de desplazamiento registrado seguiría aplicándose de forma indefinida en frames posteriores en los que el usuario ya no interactúa con la rueda.

Conclusiones y trabajo futuro

4.1. Conclusiones

Este trabajo se planteaba como el estudio y desarrollo de un conjunto de operaciones de edición para un editor de mallas tridimensionales orientado a usuarios sin conocimientos avanzados de modelado 3D. A la vista del resultado obtenido, puede considerarse que los objetivos planteados en la sección 1.2 se han alcanzado parcialmente.

Se ha estudiado el contexto de la edición de modelos tridimensionales generados a partir de contenido bidimensional, así como el panorama de herramientas de edición ya existentes, lo cual ha permitido analizar y seleccionar un conjunto de operaciones (selección, transformación, eliminación y extrusión) que cubren algunas de las necesidades más habituales de edición sencilla sobre una malla, sin requerir la amplitud de funcionalidades de una aplicación de creación 3D de propósito general. Sobre esta base, se ha diseñado e implementado una herramienta que integra dichas operaciones: importación de un modelo, visualización e interacción con la escena tridimensional mediante una cámara orbital, selección de sus elementos (vértices, aristas y polígonos) y aplicación de las operaciones de edición estudiadas, tanto mediante interacción directa con el ratón como, en el caso de la posición, mediante ajuste numérico mediante un panel de la interfaz.

El desarrollo del proyecto ha requerido, además, profundizar en aspectos propios de los gráficos: la representación interna de una malla editable a partir de un formato orientado al renderizado, la resolución de la selección de elementos en un espacio de pantalla bidimensional a partir de una escena tridimensional, o el cálculo de las transformaciones geométricas aplicadas sobre los elementos seleccionados.

Cabe señalar también que determinadas decisiones de diseño priorizan la sencillez de implementación frente a una separación de responsabilidades más estricta, como ocurre con el acoplamiento entre la lógica de edición del programa y el sistema de señales de Qt (sección 3.1.1), así como frente al rendimiento en casos de uso más exigentes que los contemplados durante el desarrollo.

4.2. Trabajo futuro

A lo largo del desarrollo de este proyecto se han identificado posibles mejoras, recogidas a continuación.

4.2.1. Rendimiento

- La deduplicación de aristas durante la carga del modelo emplea un conjunto ordenado (`std::set`), con un coste de $O(n \log n)$, en lugar de una tabla de *hash* (`std::unordered_set`), que reduciría dicho coste a $O(n)$ en promedio (sección 3.3.2.2).
- Tanto la deduplicación de vértices como la agrupación por posición geométrica emplean una comparación exacta de valores en punto flotante, sin ningún margen de tolerancia (*epsilon*), por lo que posiciones geométricamente equivalentes pero con un redondeo distinto no se reconocen como el mismo vértice lógico (sección 3.3.2.1).
- Los algoritmos de selección (*picking*) recorren linealmente la totalidad de los vértices lógicos, aristas o polígonos de la malla en cada frame, sin emplear ninguna estructura de aceleración espacial, como un octree, que permitiría descartar rápidamente los elementos alejados de la posición del ratón en mallas de mayor densidad (sección 3.3.3).
- El cálculo de las operaciones de transformación se realiza íntegramente en la CPU, y cada actualización durante un arrastre reenvía a la GPU el contenido completo del VBO de la malla, en lugar de únicamente los vértices modificados.

4.2.2. Funcionalidad

- El editor no ofrece ninguna funcionalidad de deshacer o rehacer (*undo/redo*) sobre las operaciones de edición aplicadas, una funcionalidad habitual en herramientas de este tipo.
- La distancia de extrusión está fijada en el propio código, sin admitir un valor interactivo controlado por arrastre del ratón o por el panel numérico de la interfaz, a diferencia del resto de operaciones de transformación (sección 3.3.5.3).
- El comportamiento de la eliminación de elementos es fijo, aplicando siempre una limpieza automática de la geometría que queda sin uso. Herramientas como Blender permiten al usuario decidir explícitamente cuánta geometría conectada desea conservar.
- El modelo de iluminación se limita a un término ambiente constante y un término difuso, sin componente especular, empleando una única fuente de luz direccional con dirección y color fijados en el código, sin posibilidad de configuración por parte del usuario (sección 3.3.1.2).

- El editor permite crear nueva geometría mediante extrusión, pero no ofrece ninguna operación para crear vértices completamente independientes, desconectados de la selección actual. En consecuencia, no es posible construir una malla desde cero dentro del propio editor, ni añadir geometría en una posición arbitraria sin partir de un elemento ya existente.

4.2.3. Robustez

- La carga de un modelo no gestiona adecuadamente los casos de error: si el archivo especificado está corrupto o la ruta no es válida, la aplicación termina de forma abrupta en lugar de notificar el fallo al usuario (sección ??).

4.2.4. Extensiones

- El editor admite exclusivamente el formato OBJ, tanto para la importación como para la exportación de modelos. La incorporación de una biblioteca como Assimp, descartada durante el desarrollo por la sencillez que ofrecía tinyobj-loader para el alcance de este trabajo, permitiría ampliar el soporte a otros formatos de intercambio de modelos tridimensionales.

Introduction

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Conclusions and Future Work

Conclusions and future lines of work. This chapter contains the translation of Chapter 4.

Bibliografía

- AUTODESK INC. Maya overview. <https://www.autodesk.com/uk/products/maya/overview>, 2026. Accedido: 27/08/2026.
- BLENDER FOUNDATION. Blender's History. <https://www.blender.org/about/history/>, 2025. Accedido: 27/08/2026.
- BLENDER FOUNDATION. About Blender. https://docs.blender.org/manual/en/latest/getting_started/about/index.html, 2026a. Accedido: 27/08/2026.
- BLENDER FOUNDATION. Deleting and Dissolving. <https://docs.blender.org/manual/en/latest/modeling/meshes/editing/mesh/delete.html>, 2026b. Accedido: 27/08/2026.
- FERREIRA, G., COSTA, K. y ABÍLIO, M. History of Visual Computing. <https://visgraf.github.io/history-CG/timeline-cg/mit/index.html>, 2025. Accedido: 27/08/2026.
- G-TRUC. glm. <https://glm.g-truc.net/0.9.9/index.html>, 2026. Accedido: 27/08/2026.
- KHRONOS GROUP. History of OpenGL. https://wikis.khronos.org/opengl/History_of_OpenGL, 2022. Accedido: 27/08/2026.
- KHRONOS GROUP. OpenGL. <https://www.khronos.org/opengl/>, 2026a. Accedido: 27/08/2026.
- KHRONOS GROUP. OpenGL Loading Library. https://www.khronos.org/opengl/wiki/OpenGL_Loading_Library, 2026b. Accedido: 27/08/2026.
- KREUZER, J. 3D Programming Selection Tutorial. <https://3dkingdoms.com/selection.html>, 2026. Accedido: 27/08/2026.
- QT GROUP. Qt. <https://doc.qt.io/qt-6/qt-intro.html>, 2026a. Accedido: 27/08/2026.
- QT GROUP. Qt History. https://wiki.qt.io/Qt_History, 2026b. Accedido: 27/08/2026.

RETRO REVERSING. Maya (Alias Wavefront, Autodesk) - The history of the Maya 3D software in the games industry. <https://www.retroreversing.com/maya>, 2026. Accedido: 27/08/2026.

SUTHERLAND, I. E. Sketchpad, a man-machine graphical communication system. <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-574.pdf>, 2003. Accedido: 27/08/2026.

DE VRIES, J. *Learn OpenGL: Learn Modern OpenGL Graphics Programming in a Step-by-Step Fashion*. Kendall & Wells, 2020.

DE VRIES, J. LearnOpenGL - Coordinate Systems. <https://learnopengl.com/Getting-started/Coordinate-Systems>, s.f.a. Accedido: 27/08/2026.

DE VRIES, J. LearnOpenGL - Shaders. <https://learnopengl.com/getting-started/shaders>, s.f.b. Accedido: 27/08/2026.